

1.1 多人协同开发经历

我之前有过一些协同开发的经历，主要来自程序设计以及其他计算机相关课程中的合作类作业。不过采用的协作方式比较简单，往往通过口头协商进行分工，各自完成负责的部分后，再进行汇总，并没有使用 Git 等版本控制工具进行系统的协同开发。

1.2 为什么 Git 要设计“暂存—提交”两个步骤？

Git 将暂存和提交分为两个步骤，可以让用户在正式提交前选择哪些修改需要被记录。实际开发中，我们可能同时修改多个文件，而这些修改不一定属于同一个任务。通过 `git add`，可以先将本次希望提交的修改加入暂存区，再通过 `git commit` 将暂存区中的内容形成一次提交。这样可以使每次提交的内容更加明确，也方便之后查看和管理版本历史。

1.3 `git branch` 和 `git branch -a` 的区别

`git branch` 用于查看本地分支，同时会用 `*` 标记当前所在的分支。

`git branch -a` 中的 `-a` 表示 `--all`，除了显示本地分支外，还会显示远程跟踪分支。因此，当仓库连接了 GitHub 等远程仓库时，可以使用 `git branch -a` 查看更加完整的分支情况。

2. 完成 `main.c` 文件中的 TODO 部分并进行一次 `commit`。

```
fanmo@LAPTOP-R7Q6LOMR:~/ICS/Lab0-GitLab$ git-log
* d55aeca (HEAD -> main) add a sentence
* cb395a7 (origin/main, origin/HEAD) Initial commit
```

3.1 Commit Message 规范

文章介绍了规范编写 Commit Message 的意义和方法。简而言之，规范的提交信息能够提供更多的历史信息，方便快速浏览；过滤某些 commit（比如文档改动），便于快速查找信息；同时可以直接从 commit 生成 Change log。

文章以 Angular 规范为例，将 Commit Message 分为 Header、Body 和 Footer，其中 Header 通过 type、scope 和 subject 等信息简要说明一次提交的类型、影响范围和具体内容。例如，可以使用 `feat` 表示新增功能、`fix` 表示修复问题、`docs` 表示修改文档。通过统一的提交规范，可以使项目的版本历史更加清晰，也有利于多人协作和后续维护。

3.2 语义化版本

文章介绍了语义化版本控制的规则，其基本形式为 `X.Y.Z`，分别代表主版本号、次版本号和修订号。当软件进行了不兼容的 API 修改时增加主版本号；增加向下兼容的新功能时增加次版本号；进行向下兼容的问题修复时增加修订号。

例如从 1.2.3 更新到 1.2.4，通常意味着进行了兼容的问题修复，而更新到 2.0.0 往往意味着版本大更新，产生了不兼容的 API 修改，这时用户往往需要重新下载软件安装包以获得更新。这样的版本规范使版本号本身能够反映软件发生了什么类型的变化，从而帮助开发者管理软件版本和项目依赖，降低版本混乱和依赖冲突带来的问题。

3.3 为什么要学习 Git

阅读这两篇文章后，我意识到 Git 最核心的功能就是通过版本控制和一些规范，有序地管理一个不断变化的项目。Commit Message 可以记录每次修改的目的，Git 的提交历史可以保存项目的发展过程，而语义化版本又能够进一步描述

不同版本之间的变化。在多人协作时，这些信息可以帮助不同开发者了解其他人进行了哪些修改，并在出现问题时追踪和恢复之前的版本。因此，相比简单地由人员各自完成任务后再汇总，使用 Git 能够使项目的修改、协作和版本管理更加清晰和规范。

4. Git 分支管理

(1) 建立 feature 分支，修改 main.c

```
fanmo@LAPTOP-R7Q6LOMR:~/ICS/Lab0-GitLab$ git branch
* main
fanmo@LAPTOP-R7Q6LOMR:~/ICS/Lab0-GitLab$ git checkout -b feature
Switched to a new branch 'feature'
fanmo@LAPTOP-R7Q6LOMR:~/ICS/Lab0-GitLab$ vi main.c
fanmo@LAPTOP-R7Q6LOMR:~/ICS/Lab0-GitLab$ git add .

fanmo@LAPTOP-R7Q6LOMR:~/ICS/Lab0-GitLab$ git commit -m "add a sentence in branch feature."
[feature fa64a85] add a sentence in branch feature.
1 file changed, 1 insertion(+)
```

(2) 回到 main 分支，修改 main.c

```
fanmo@LAPTOP-R7Q6LOMR:~/ICS/Lab0-GitLab$ git checkout main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)
fanmo@LAPTOP-R7Q6LOMR:~/ICS/Lab0-GitLab$ vi main.c
fanmo@LAPTOP-R7Q6LOMR:~/ICS/Lab0-GitLab$ git add .
fanmo@LAPTOP-R7Q6LOMR:~/ICS/Lab0-GitLab$ git commit -m "add a sentence in branch main."
[main eadd27b] add a sentence in branch main.
1 file changed, 1 insertion(+)
fanmo@LAPTOP-R7Q6LOMR:~/ICS/Lab0-GitLab$ git-log
* eadd27b (HEAD -> main) add a sentence in branch main.
| * fa64a85 (feature) add a sentence in branch feature.
|/
* d55aeca add a sentence
* cb395a7 (origin/main, origin/HEAD) Initial commit
```

(3) 合并分支，消除冲突

```
fanmo@LAPTOP-R7Q6LOMR:~/ICS/Lab0-GitLab$ git merge feature
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
fanmo@LAPTOP-R7Q6LOMR:~/ICS/Lab0-GitLab$ vi main.c
fanmo@LAPTOP-R7Q6LOMR:~/ICS/Lab0-GitLab$ git add .
fanmo@LAPTOP-R7Q6LOMR:~/ICS/Lab0-GitLab$ git commit
[main 378095f] Merge branch 'feature'
fanmo@LAPTOP-R7Q6LOMR:~/ICS/Lab0-GitLab$ git-log
* 378095f (HEAD -> main) Merge branch 'feature'
| \
| * fa64a85 (feature) add a sentence in branch feature.
* | eadd27b add a sentence in branch main.
|/
* d55aeca add a sentence
* cb395a7 (origin/main, origin/HEAD) Initial commit
```

此处消除冲突的方法是通过直接编辑合并后的 main.c 文件，比如我就直接删去了合并后 feature 分支带来的更改，仅保留 main 分支产生的变化。